reading tests. Testing frameworks often hide these things behind a convenient DSL that makes sense in a testing domain:

```
expect('fundamental').to.contain 'fun'
```

This is a minor shift, and for this example it's actually more verbose. So why do it? It makes life easier for users. The more control you have over the language, the more power you have to create with it.

The most common approach to creating DSLs in CoffeeScript comes from JavaScript, and that's to use a fluent interface. You don't have to use a fluent interface, though, because CoffeeScript syntax is sparse.

8.2.3. Object literals

With very little in the way of syntax, CoffeeScript object literals are a useful tool for creating internal DSLs. Imagine Agtron standing next to your desk, coffee in hand. "There's a law," he states, "known as Zawinski's law, that 'Every program attempts to expand until it can read mail.'" If you'll eventually need to write a program that deals with email, you might as well get some practice now.

Consider the format of the exchange between a client and a server using the Simple Mail Transfer Protocol (SMTP). In the following example, client requests are in bold and server responses are in regular-weight font:

```
HELO coffeescriptinaction.com
250 OK
MAIL FROM: scruffy@coffeescriptinaction.com
250 OK - mail from <scruffy@coffeescriptinaction.com>
RCPT TO: agtron@coffeescriptinaction.com
250 OK - Recipient <agtron@coffeescriptinaction.com>
DATA
354 Send data.  End with CRLF.CRLF
Hi Agtron. Just Scruffy testing SMTP.
250 OK QUIT
```

An example of communication between a server and client using SMTP

You *might* implement your SMTP library by emulating the protocol: